

Gestión de Biblioteca

Tabla de contenidos

1	Vamos a desarrollar el proyecto paso a paso	1
1.0.1	Paso 1: Crear los Servicios	1
1.0.2	Paso 2: Refactorizar los Controladores	5
1.0.3	Resumen	6

1 Vamos a desarrollar el proyecto paso a paso

Vamos a refactorizar el proyecto para usar servicios en lugar de manejar toda la lógica en los controladores. A medida que avanzamos, también terminaremos el **controlador de préstamos** y la lógica para **devoluciones tardías**.

Voy a dividir el proceso en pasos claros:

1. **Crear los servicios:** Definiremos los servicios para cada entidad (`LibroService`, `AutorService`, `UsuarioService`, `PrestamoService`).
2. **Refactorizar los controladores:** Actualizaremos cada controlador para que delegue la lógica a los servicios.
3. **Implementar la lógica de devoluciones tardías:** Añadiremos esta lógica en `PrestamoService` y actualizaremos `PrestamoController` para manejar devoluciones y calcular si una devolución es tardía.

1.0.1 Paso 1: Crear los Servicios

- 1.1. **LibroService** Este servicio manejará la lógica de negocio de los libros, como crear, actualizar, eliminar y buscar libros. También implementaremos métodos de paginación y filtro.

```
import com.biblioteca.gestion.entities.Libro;
import com.biblioteca.gestion.repositories.LibroRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

import java.util.Optional;
```

```

@Service
public class LibroService {

    @Autowired
    private LibroRepository libroRepository;

    public Page<Libro> listarLibros(String titulo, String genero,
                                    Pageable pageable) {

        if (titulo != null) {
            return libroRepository.findByTituloContaining(titulo, pageable);
        } else if (genero != null) {
            return libroRepository.findByGenero(genero, pageable);
        } else {
            return libroRepository.findAll(pageable);
        }
    }

    public Optional<Libro> obtenerLibro(Long id) {
        return libroRepository.findById(id);
    }

    public Libro crearLibro(Libro libro) {
        return libroRepository.save(libro);
    }

    public Optional<Libro> actualizarLibro(Long id, Libro libroActualizado) {
        return libroRepository.findById(id).map(libro -> {
            libro.setTitulo(libroActualizado.getTitulo());
            libro.setGenero(libroActualizado.getGenero());
            libro.setAñoPublicacion(libroActualizado.getAñoPublicacion());
            libro.setEstado(libroActualizado.getEstado());
            libro.setAutor(libroActualizado.getAutor());
            return libroRepository.save(libro);
        });
    }

    public boolean eliminarLibro(Long id) {
        if (libroRepository.existsById(id)) {
            libroRepository.deleteById(id);
            return true;
        }
        return false;
    }
}

```

- 1.2. **AutorService** El servicio de autores se encarga de manejar las operaciones CRUD para la entidad `Autor`.

```
import com.biblioteca.gestion.entities.Autor;
import com.biblioteca.gestion.repositories.AutorRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class AutorService {

    @Autowired
    private AutorRepository autorRepository;

    public List<Autor> listarAutores() {
        return autorRepository.findAll();
    }

    public Autor crearAutor(Autor autor) {
        return autorRepository.save(autor);
    }

    public Optional<Autor> actualizarAutor(Long id, Autor autorActualizado) {
        return autorRepository.findById(id).map(autor -> {
            autor.setNombre(autorActualizado.getNombre());
            autor.setNacionalidad(autorActualizado.getNacionalidad());
            return autorRepository.save(autor);
        });
    }

    public boolean eliminarAutor(Long id) {
        if (autorRepository.existsById(id)) {
            autorRepository.deleteById(id);
            return true;
        }
        return false;
    }
}
```

- 1.3. **UsuarioService** Este servicio manejará la lógica relacionada con los usuarios.

```
import com.biblioteca.gestion.entities.Usuario;
import com.biblioteca.gestion.repositories.UsuarioRepository;
import org.springframework.beans.factory.annotation.Autowired;
```

```

import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class UsuarioService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    public Usuario registrarUsuario(Usuario usuario) {
        return usuarioRepository.save(usuario);
    }

    public Optional<Usuario> obtenerUsuario(Long id) {
        return usuarioRepository.findById(id);
    }

    public Optional<Usuario> obtenerUsuarioPorEmail(String email) {
        return usuarioRepository.findByEmail(email);
    }
}

```

- 1.4. **PrestamoService** (Incluyendo la Lógica de Devoluciones Tardías) Aquí es donde implementamos la lógica de los préstamos, incluyendo la verificación de **devoluciones tardías**.

```

import com.biblioteca.gestion.entities.Libro;
import com.biblioteca.gestion.entities.Prestamo;
import com.biblioteca.gestion.repositories.LibroRepository;
import com.biblioteca.gestion.repositories.PrestamoRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

@Service
public class PrestamoService {

    @Autowired
    private PrestamoRepository prestamoRepository;

    @Autowired
    private LibroRepository libroRepository;
}

```

```

    public Prestamo registrarPrestamo(Prestamo prestamo) {
        Optional<Libro> libro = libroRepository.findById(prestamo.getLibro().getId());
        if (libro.isPresent() && "disponible".equalsIgnoreCase(libro.get().getEstado())) {
            libro.get().setEstado("prestado");
            prestamo.setFechaPrestamo(LocalDate.now());
            return prestamoRepository.save(prestamo);
        }
        throw new IllegalArgumentException("El libro no está disponible para prestar");
    }

    public Optional<Prestamo> devolverLibro(Long prestamoId) {
        return prestamoRepository.findById(prestamoId).map(prestamo -> {
            prestamo.setFechaDevolucion(LocalDate.now());
            Libro libro = prestamo.getLibro();
            libro.setEstado("disponible");
            // Verificar si la devolución es tardía
            if (prestamo.getFechaPrestamo().plusDays(14).isBefore(LocalDate.now())) {
                prestamo.setDevolucionTardia(true); // Marcar devolución tardía
            }
            prestamoRepository.save(prestamo);
            return prestamo;
        });
    }

    public List<Prestamo> listarPrestamosPorUsuario(Long usuarioId) {
        return prestamoRepository.findByUsuarioId(usuarioId);
    }
}

```

1.0.2 Paso 2: Refactorizar los Controladores

Ahora, actualizaremos los controladores para delegar la lógica en estos servicios. Por ejemplo, `PrestamoController`:

```

import com.biblioteca.gestion.entities.Prestamo;
import com.biblioteca.gestion.services.PrestamoService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/prestamos")
public class PrestamoController {

```

```

@Autowired
private PrestamoService prestamoService;

@PostMapping
public ResponseEntity<Prestamo> registrarPrestamo(@RequestBody Prestamo prest
    try {
        Prestamo nuevoPrestamo = prestamoService.registrarPrestamo(prestamo);
        return ResponseEntity.ok(nuevoPrestamo);
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(null);
    }
}

@PutMapping("/{id}/devolver")
public ResponseEntity<Prestamo> devolverLibro(@PathVariable Long id) {
    return prestamoService.devolverLibro(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

@GetMapping("/usuario/{usuarioId}")
public ResponseEntity<List<Prestamo>> listarPrestamosPorUsuario(@PathVariable
    List<Prestamo> prestamos = prestamoService.listarPrestamosPorUsuario(usua
    return ResponseEntity.ok(prestamos);
}
}

```

1.0.3 Resumen

1. **Servicios creados** para Libro, Autor, Usuario, y Prestamo, encapsulando la lógica de negocio.
2. **Controladores refactorizados** para delegar en los servicios.
3. **Lógica de devoluciones tardías** implementada en PrestamoService y gestionada desde PrestamoController.

Con estos cambios, la estructura es más clara y permite una gestión de lógica de negocio eficiente y separada de la capa de controladores. ¡Listo para revisar cualquier duda o continuar!